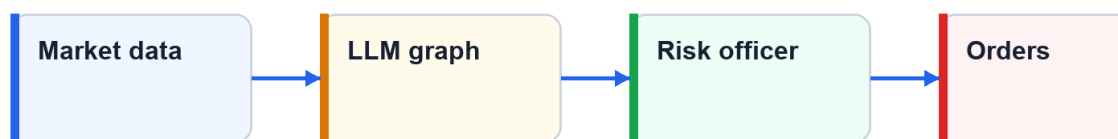


Как мы строили автономного LLM-агента для MOEX AI Hackathon

LLM trading agent

Autonomous MOEX sandbox trading system: data, model, risk layer and execution.



Main lesson: the model proposes, deterministic controls decide

Схема: рыночные данные проходят через LLM-граф, риск-слой и исполнение заявок

MOEX AI Hackathon был соревнованием не про "прикрутить чат к котировкам". Участникам нужно было собрать автономного агента, который сам анализирует рынок, управляет портфелем и торгует в sandbox-контуре.

Мы быстро пришли к простой архитектурной формуле:

LLM формирует торговое намерение. Risk manager решает, можно ли его исполнять.

В статье рассказываем, как выглядел наш контур: данные, LLM-граф, live portfolio context, риск-слой, ArenaGo API, журнал решений и Grafana. Без инвестиционных советов и без обещаний доходности: это инженерный разбор решения для хакатона.

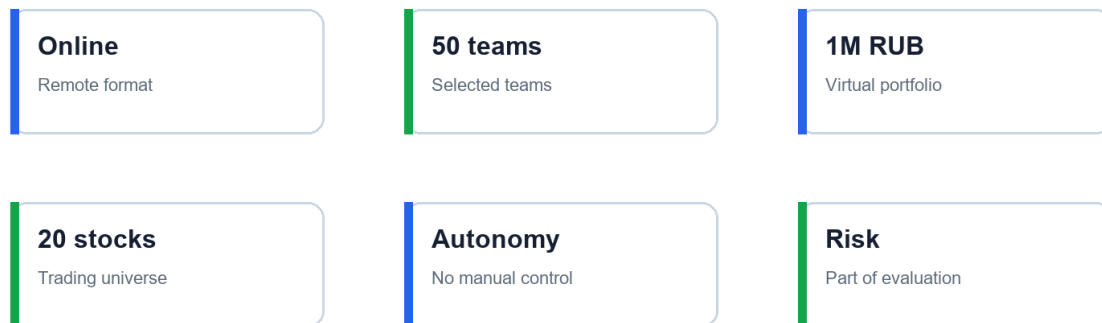
Контекст хакатона

MOEX AI Hackathon проходил онлайн. На [официальном сайте](#) указан общий период 13 мая - 30 июня, призовой фонд 1 млн рублей и задача: создать ИИ-агента для биржевой торговли. Каждая команда получала 1 млн виртуальных рублей на sandbox-счете.

По [пресс-релизу Московской биржи](#), в отбор прошли 50 команд. Оценивались не только результаты портфеля, но и архитектура, автономность, прозрачность логики, риск-менеджмент, работа с данными, оборот и итоговый капитал.

По [новости на Хабре](#), агенты торговали набором из 20 акций. Там же и в [пересказе Gazeta](#) хорошо виден важный для нас вывод: большое количество сделок само по себе не гарантирует хороший результат.

MOEX AI Hackathon: constraints



The score depended not only on PnL, but also on autonomy, architecture, risk and turnover

Ключевые условия хакатона: онлайн-формат, sandbox-портфель, 20 акций, автономность и риск

На уровне задачи все выглядит линейно:

1. Получить рыночные данные.
2. Принять решение BUY / SELL / HOLD.
3. Отправить заявку.
4. Увеличить стоимость портфеля.

На уровне реализации появляется другой список:

- данные могут быть устаревшими;
- модель может вернуть неполный JSON;
- позиция по тикеру уже может быть открыта;
- SELL может быть закрытием long или открытием short;
- BUY может быть открытием long или закрытием short;
- комиссия делает мелкие движения неинтересными;
- частая торговля может выполнить turnover, но ухудшить результат;
- внешние API могут отвечать долго или нестабильно.

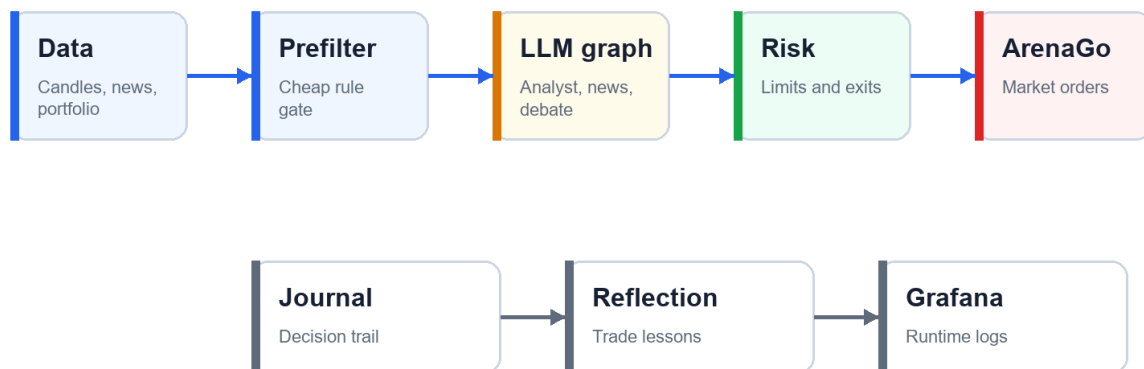
Поэтому мы строили не одну "модель-трейдер", а контур принятия решений.

Общая архитектура

Один торговый тик у нас состоял из нескольких фаз:

1. Scheduler получает список тикеров, рыночный контекст, текущий портфель и сделки за день.
2. Для каждого тикера собираются свечи, признаки, новости и состояние позиции.
3. LLM-граф формирует торговое намерение.
4. Risk manager проверяет намерение и может пропустить, урезать, заблокировать или заменить его защитным выходом.
5. Execution layer отправляет заявку через ArenaGo API.
6. Journal, reflection и Grafana сохраняют след решения.

Solution architecture



Data enters on the left; every order must pass the deterministic risk layer

Архитектура: данные, prefilter, LLM-граф, risk manager, ArenaGo, журнал и мониторинг

Важное разделение: LLM не отправляет заявку напрямую. Она возвращает структурированное решение, а детерминированный слой проверяет, можно ли его исполнять.

Это позволило держать разные классы логики отдельно:

- LLM отвечает за интерпретацию контекста и формулирование гипотезы;
- risk manager отвечает за инварианты портфеля;
- scheduler отвечает за ритм и устойчивость цикла;
- journal и Grafana отвечают за наблюдаемость.

Такую систему легче отлаживать: когда сделка не прошла, видно, на каком слое это произошло.

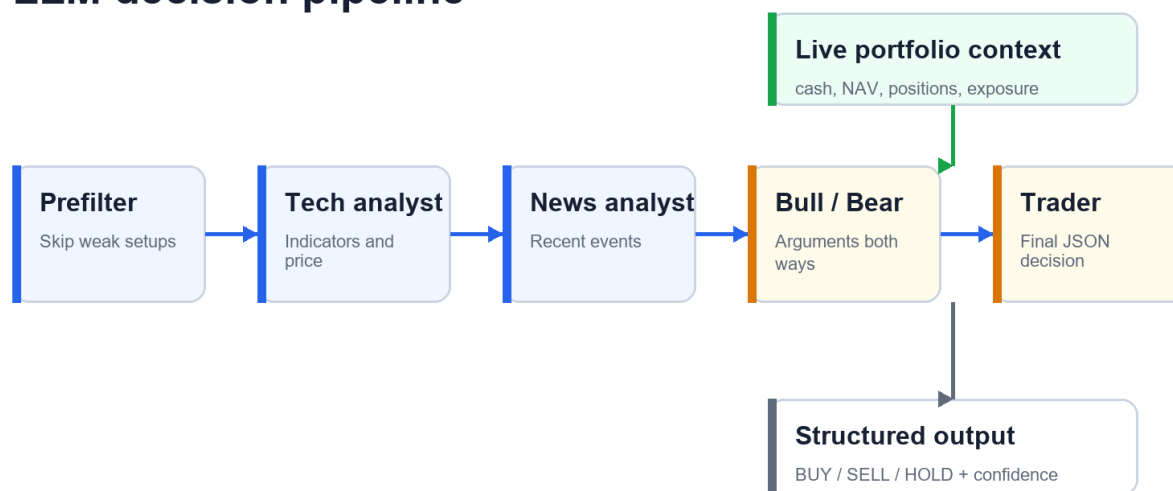
LLM-граф

Мы не делали один большой prompt. Решение проходило через несколько ролей:

- prefilter отсекает слабые ситуации до LLM-вызовов;

- technical analyst смотрит на свечи и индикаторы;
- news analyst добавляет новостной фон;
- bull/bear debate формулирует аргументы за и против;
- trader собирает финальное решение в структурированный JSON.

LLM decision pipeline



The model sees portfolio state before it recommends changing portfolio state

LLM-пайплайн: prefilter, технический анализ, новости, debate, trader и портфельный контекст

Важный момент: trader видит live portfolio context. Это не просто "дополнительная информация в промпте", а часть операционного состояния:

- cash;
- NAV;
- текущие позиции;
- средняя цена позиции;
- gross и net exposure;
- вес тикера в портфеле;
- сделки за день.

Без этого модель рассуждает как аналитик: "по инструменту есть сигнал". С портфельным контекстом она начинает рассуждать ближе к управляющему: "у нас уже есть позиция, что произойдет с портфелем, если ее изменить".

Почему live portfolio context оказался критичным

В торговом агенте одно и то же действие может означать разные вещи.

Если у нас нет позиции, SELL может быть открытием short. Если у нас long, SELL может быть закрытием позиции. Если у нас short, BUY может быть cover. Если у нас нет позиции, BUY может

быть открытием long.

Для человека это очевидно. Для модели это очевидно только если состояние портфеля явно передано в контекст и если risk manager правильно классифицирует операцию.

После добавления live portfolio context trader начал получать не абстрактный вопрос "что делать с тикером?", а конкретный вопрос "как изменить текущее состояние портфеля?". Это снижает риск независимых, несвязанных друг с другом решений.

Данные и проблема stale candles

Один из существенных инцидентов был связан не с LLM, а с данными.

Мы переходили на 10-минутные свечи, чтобы агент работал на более коротком горизонте. При этом поймали проблему с устаревшими свечами из-за пагинации и окна истории. Система формально получала данные, индикаторы считались, LLM давала уверенный ответ, но рыночный snapshot мог уже не соответствовать текущему состоянию.

Фикс был не в промпте. Мы добавили проверку свежести данных и поправили получение свечей. Если данные устарели, тикер не должен торговаться.

Отдельный урок: смена таймфрейма меняет не только AGENT_INTERVAL. При переходе на 10m пришлось пересматривать:

- окно свечей;
- prefilter;
- масштаб волатильности;
- expected value horizon;
- min edge;
- early exit.

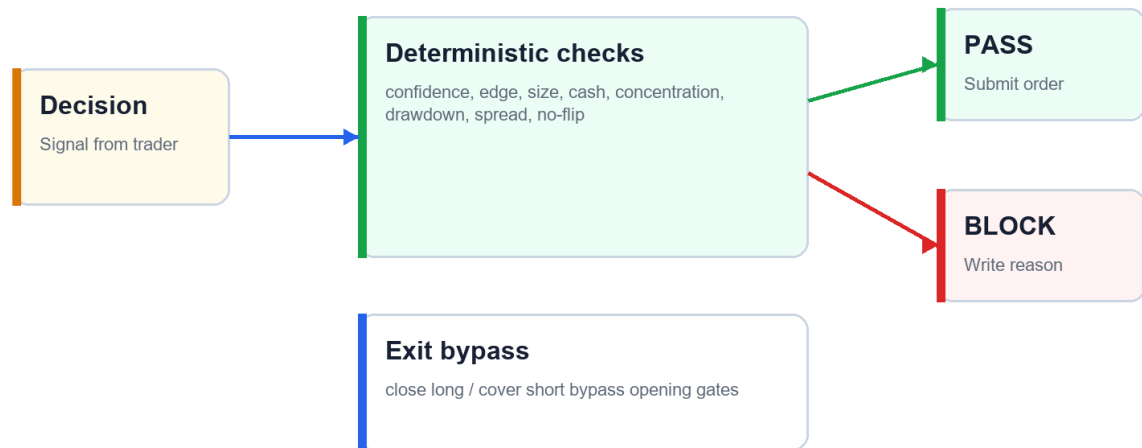
Порог, разумный для 60m, может почти полностью остановить торговлю на 10m.

Risk manager

Risk manager - отдельный детерминированный слой. Он получает решение модели и контекст портфеля, а затем возвращает verdict:

- pass;
- block;
- clip size;
- force close;
- take profit;
- stop loss.

Risk officer



Opening risk is strict; de-risking exits have a shorter path

Risk manager: решение модели проходит через проверки, после чего заявка либо исполняется, либо блокируется

Проверки включали:

- confidence;
- min edge с учетом волатильности и комиссии;
- лимит веса инструмента;
- секторные ограничения;
- cash buffer;
- daily loss и drawdown;
- spread;
- no-flip;
- take-profit и stop-loss.

Самая важная деталь: закрытие позиции не должно проходить те же проверки, что и открытие новой.

Если агент в long и продает, это может быть снижение риска. Если агент в short и покупает, это может быть cover. Такие операции нельзя блокировать по тем же low-confidence gates, которые применяются к открытию новой позиции.

Мы зафиксировали правило:

Opening risk проверяется строго. De-risking exits должны проходить везде, где это безопасно.

Это правило особенно важно для short-логики, потому что направление заявки само по себе не описывает экономический смысл операции.

Overtrading

На хакатоне был оборотный floor, поэтому агент не мог просто ничего не делать. Но другая крайность - слишком частая торговля.

Частая торговля создает несколько проблем:

- комиссия начинает съедать мелкие движения;
- модель может реагировать на шум;
- позиция может часто закрываться и переоткрываться;
- один и тот же сигнал может исполняться несколько раз;
- портфель становится трудно объяснять.

Мы добавили несколько ограничений:

- no-flip: противоположный сигнал сначала закрывает позицию до flat, а не сразу переворачивает ее;
- commission-aware edge: ожидаемое движение должно покрывать издержки;
- turnover monitoring: в логах видно, как агент идет к целевому обороту;
- stricter low-confidence gates для открытия риска;
- fixed take-profit / stop-loss как deterministic overrides.

В итоге risk layer стал не просто блокировщиком, а регулятором поведения агента.

Structured output

Свободный текст от LLM неудобен для торговой системы. Нужен контракт.

Мы использовали структурированные ответы:

- signal: BUY / SELL / HOLD;
- size;
- confidence;
- reasoning;
- risk notes.

Если модель не возвращает валидную структуру, это не частично успешное решение, а отсутствие решения. Такой тикер должен уйти в no-op/error, а не исполняться по догадке.

Под конец мы поймали характерный edge case: fallback JSON вернул HOLD без confidence. Для человека это выглядит допустимо, но для схемы это ошибка. Мы добавили безопасный default, чтобы HOLD без confidence не останавливал цикл.

Вывод простой: в автономной системе даже небольшое поле схемы может стать production-инцидентом.

Timeouts и внешние API

Торговый цикл зависит от нескольких внешних систем:

- источники рыночных данных;
- LLM API;
- ArenaGo API;
- мониторинг и логирование.

Любая из них может отвечать долго. Поэтому у нас появились per-ticker timeout, retry-логика и fallback-поведение. Если один тикер завис, он не должен блокировать весь тик. Если LLM не успела вернуть решение, лучше пропустить сделку, чем исполнить просроченное намерение.

Отдельно мы убрали webhook-логирование как обязательный контур. Внешний webhook быстро превращался в источник шума и 429. Для production-наблюдаемости надежнее использовать Loki/Grafana и локальный журнал.

Наблюдаемость

Для автономного агента важно не только "что он сделал", но и "почему".

Мы логировали:

- market context;
- portfolio snapshot;
- структурированное решение модели;
- risk verdict;
- причину block или clip;
- отправленную заявку;
- ответ ArenaGo;
- turnover pace;
- ошибки и таймауты.

Многие фиксы появились после анализа логов. Например, по логам было видно:

- почему агент слишком часто торгует;
- где risk manager режет решения;
- когда модель пытается закрыть позицию, а система трактует это как opening;
- когда данные устарели;
- почему конкретный тикер зависает на обработке.

Без такой трассировки автономный агент быстро становится трудноуправляемым.

Основные повороты в разработке

Мы не стали выносить в статью бытовой путь от первого тестового ордера. Полезнее показать системные повороты.

Engineering timeline



Pattern

Most fixes were not prompt tweaks. They fixed boundaries: data freshness, portfolio context, risk gates and observability.

The useful story is not the first launch; it is the sequence of system-level fixes

Timeline: данные, NAV, risk layer, 10m bars, no-flip, logs и финальное hardening

Крупные изменения выглядели так:

1. Перестали опираться на недоступные или неверно интерпретированные поля брокера и аккуратнее посчитали NAV.
2. Добавили LLM-бэкстест, чтобы проверять не только техническую стратегию, но и реальный граф решений.
3. Ввели no-flip и fixed TP/SL для снижения churn.
4. Добавили Algorack flow, prefilter и reflection.
5. Добавили EV-дисциплину и turnover monitoring.
6. Поймали stale candles на 10m, поправили получение данных и вернули 10m осознанно.
7. Передали trader-y live portfolio context.
8. Убрали webhook-spam и усилили устойчивость к зависаниям API.
9. Добавили fast/mid market context, а затем убрали directional bias: контекст должен информировать, а не подталкивать.
10. Закрыли structured-output edge case с отсутствующим confidence.

Большинство важных изменений были не "улучшением промпта", а уточнением границ между слоями.

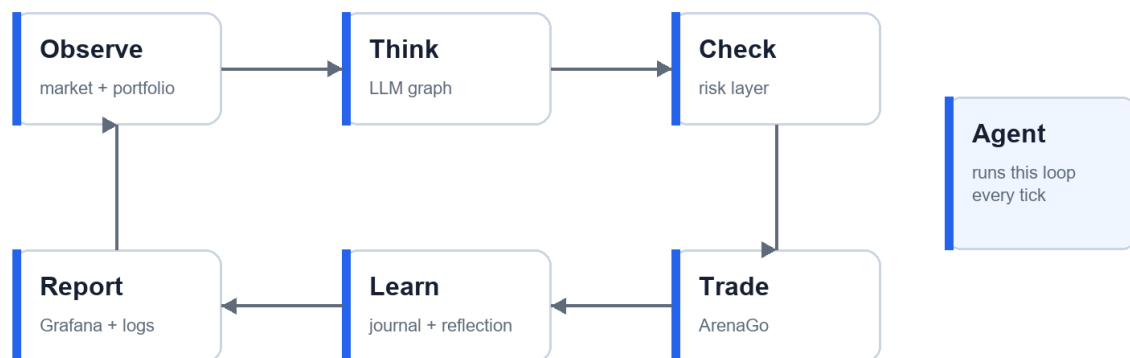
Финальный production loop

Финальная версия агента работала как цикл:

1. Observe: получить рынок и портфель.
2. Think: прогнать LLM-граф.

3. Check: проверить решение risk layer.
4. Trade: отправить заявку.
5. Learn: записать journal/reflection.
6. Report: отдать состояние в логи и мониторинг.

Production loop



A trading agent is a loop, not a single model call

Production loop: observe, think, check, trade, learn, report

Внутри этого цикла разные задачи решались разными инструментами. Быстрые LLM-роли использовались для analyst/news/debate, более сильная модель - для финального trader-синтеза. Risk manager не использовал LLM. Scheduler и observability были отдельными слоями.

Такой подход оказался устойчивее, чем попытка решить все одним запросом к модели.

Что мы вынесли

Главный вывод: торговый LLM-агент - это не модель, которая "угадывает рынок". Это система принятия решений.

LLM полезна там, где нужно собрать контекст, сформулировать гипотезу и объяснить reasoning. Но право исполнить заявку должно проходить через слой, который не зависит от стиля ответа модели.

Практические выводы:

- портфельный контекст обязателен;
- данные должны проверяться на свежесть;
- structured output должен быть строгим;
- exits нельзя блокировать как openings;
- комиссия должна участвовать в min edge;

- overtrading нужно контролировать отдельно;
- observability является частью стратегии, а не только DevOps-надстройкой.

Для нас MOEX AI Hackathon стал проверкой не столько качества конкретного прогноза, сколько архитектуры автономного агента. И главный инженерный урок оказался простым: чем больше свободы получает LLM, тем сильнее должен быть контур проверяемых ограничений вокруг нее.

Ссылки

- [MOEX AI Hackathon, официальный сайт](#)
- [Московская биржа подвела итоги MOEX AI Hackathon](#)
- [Хабр: первые результаты автономных торгов MOEX AI Hackathon](#)
- [Gazeta: большое число сделок не гарантировало прибыль](#)

Примечание перед публикацией

Изображения лежат в папке images/. Для публикации на Хабре их нужно загрузить через редактор Хабра и заменить локальные ссылки на URL загруженных файлов. В тексте нет токенов, приватных ссылок и инвестиционных рекомендаций.